

JAVA PROGRAMMING MASTERCLASS

壓縮與解壓縮檔案

「用 `java.util.zip` 打包與解開你的檔案」

[← 返回目錄](#)

章節大綱

#	節次	主題
1	23-1	java.util.zip 套件概觀
2	23-1	核心類別介紹
3	23-2	壓縮單一檔案
4	23-2	壓縮多個檔案
5	23-2	壓縮整個目錄
6	23-3	用 ZipInputStream 解壓縮
7	23-3	用 ZipFile 讀取 ZIP
8	現代寫法	NIO + ZIP File System

23-1

java.util.zip 套件概觀

java.util.zip 套件架構

1	java.util.zip	
2	├── ZipEntry	← 代表 ZIP 內的一個檔案或目錄項目
3	├── ZipOutputStream	← 寫入 (壓縮) ZIP 資料
4	├── ZipInputStream	← 讀取 (解壓縮) ZIP 資料 (循序)
5	├── ZipFile	← 讀取 ZIP 檔案 (支援隨機存取)
6	├── Deflater	← 底層壓縮引擎 (DEFLATE 演算法)
7	├── Inflater	← 底層解壓縮引擎
8	├── GZIPOutputStream	← 寫入 .gz 格式
9	├── GZIPInputStream	← 讀取 .gz 格式

💡 **ZIP vs GZIP** : ZIP 可打包多個檔案 ; GZIP 只壓縮單一串流 , 通常配合 tar 使用 (.tar.gz) 。

ZIP 檔案結構概念

```
1  archive.zip
2  |— file1.txt      ← ZipEntry (檔案)
3  |— images/       ← ZipEntry (目錄, name 結尾有 /)
4  |   |— logo.png  ← ZipEntry (檔案)
5  |   |— banner.jpg ← ZipEntry (檔案)
6  |— data/
7  |   |— report.csv ← ZipEntry (檔案)
```

 **ZipEntry** 是關鍵：每個壓縮項目（檔案或資料夾）都用一個 **ZipEntry** 物件表示，包含名稱、大小、壓縮大小、修改時間等後設資料（metadata）。

ZipEntry 核心方法

方法	說明
<code>getName()</code>	取得 entry 名稱 (含路徑)
<code>getSize()</code>	原始未壓縮大小 (bytes) ，未知則 -1
<code>getCompressedSize()</code>	壓縮後大小 (bytes) ，未知則 -1
<code>getCrc()</code>	CRC-32 校驗碼，未知則 -1
<code>getTime()</code>	修改時間 (毫秒) ，未設則 -1
<code>isDirectory()</code>	是否為目錄 (名稱是否以 <code>/</code> 結尾)
<code>setMethod(int)</code>	設定壓縮方式： <code>DEFLATED</code> 或 <code>STORED</code>

ZipOutputStream 核心方法

方法	說明
<code>putNextEntry(ZipEntry e)</code>	開始寫入一個新的 entry
<code>write(byte[], int, int)</code>	寫入目前 entry 的資料
<code>closeEntry()</code>	關閉目前 entry，準備下一個
<code>setLevel(int level)</code>	壓縮等級 0 (不壓) ~ 9 (最高)，預設 -1
<code>setMethod(int method)</code>	預設壓縮方式： <code>DEFLATED</code> 或 <code>STORED</code>
<code>setComment(String comment)</code>	設定整個 ZIP 檔案的備註
<code>finish()</code>	完成寫入但不關閉底層串流

ZipInputStream vs ZipFile

比較項目	ZipInputStream	ZipFile
存取方式	循序 (Sequential)	隨機 (Random access)
資料來源	任何 InputStream (網路、記憶體)	磁碟上的實體檔案
取得特定 entry	必須逐一掃描	getEntry(name) 直接取得
平行讀取多個 entry	不支援	支援
適用場景	串流下載、管線處理	本機已存在的 ZIP 檔案

💡 選用建議：有本機 ZIP 檔案且需要查詢特定 entry → 用 **ZipFile**；處理串流或不確定來源 → 用 **ZipInputStream**。

23-2

壓縮 (Zip) 檔案

最簡壓縮範例：單一檔案

```
1  try (ZipOutputStream zos =  
2      new ZipOutputStream(new FileOutputStream("hello.zip"))) {  
3  
4      zos.putNextEntry(new ZipEntry("hello.txt"));  
5      byte[] data = Files.readAllBytes(Path.of("hello.txt"));  
6      zos.write(data);  
7      zos.closeEntry();  
8  }
```

✓ 三步驟口訣：**putNextEntry** (宣告 entry) → **write** (寫資料) → **closeEntry** (結束 entry)

壓縮單一檔案 – 完整版（含緩衝）

```
1  public static void zipSingleFile(String src, String dest)
2      throws IOException {
3      byte[] buffer = new byte[4096];
4      try (FileOutputStream fos = new FileOutputStream(dest);
5           ZipOutputStream zos = new ZipOutputStream(fos);
6           FileInputStream fis = new FileInputStream(src)) {
7
8          zos.putNextEntry(new ZipEntry(new File(src).getName()));
9          int len;
10         while ((len = fis.read(buffer)) > 0) {
11             zos.write(buffer, 0, len);
12         }
13         zos.closeEntry();
14     }
15 }
```

壓縮多個檔案

```
1  public static void zipMultipleFiles(  
2      List<String> srcFiles, String dest) throws IOException {  
3      byte[] buf = new byte[4096];  
4      try (ZipOutputStream zos =  
5          new ZipOutputStream(new FileOutputStream(dest))) {  
6          for (String src : srcFiles) {  
7              try (FileInputStream fis = new FileInputStream(src)) {  
8                  zos.putNextEntry(new ZipEntry(new File(src).getName  
9  (())));  
10                 int len;  
11                 while ((len = fis.read(buf)) > 0)  
12                     zos.write(buf, 0, len);  
13                 zos.closeEntry();  
14             }  
15         }  
16     }
```


壓縮多個檔案 – 呼叫範例

```
1 public static void main(String[] args) throws IOException {  
2     List<String> files = List.of(  
3         "report.txt",  
4         "data.csv",  
5         "image.png"  
6     );  
7     zipMultipleFiles(files, "archive.zip");  
8     System.out.println("壓縮完成：archive.zip");  
9 }
```

💡 設定壓縮等級：在第一個 putNextEntry 前加 `zos.setLevel(9)` 可啟用最高壓縮比（速度較慢）；`setLevel(0)` 等同不壓縮。

壓縮整個目錄 – 遞迴輔助方法

```
1  private static void addDirToZip(File dir, String base,  
2                                ZipOutputStream zos)  
3      throws IOException {  
4      for (File file : dir.listFiles()) {  
5          String entryName = base + file.getName();  
6          if (file.isDirectory()) {  
7              zos.putNextEntry(new ZipEntry(entryName + "/"));  
8              zos.closeEntry();  
9              addDirToZip(file, entryName + "/", zos);  
10         } else {  
11             zos.putNextEntry(new ZipEntry(entryName));  
12             Files.copy(file.toPath(), zos);  
13             zos.closeEntry();  
14         }  
15     }  
16 }
```

壓縮整個目錄 – 呼叫入口

```
1  public static void zipDirectory(String srcDir, String dest)
2      throws IOException {
3      File dir = new File(srcDir);
4      try (ZipOutputStream zos =
5          new ZipOutputStream(new FileOutputStream(dest))) {
6          addDirToZip(dir, dir.getName() + "/", zos);
7      }
8  }
9
10 // 呼叫方式
11 zipDirectory("myProject", "myProject.zip");
```

⚠ 注意 `NullPointerException` : `dir.listFiles()` 在目錄不存在或無讀取權限時會回傳 `null` , 實務中應先做 `null` 檢查。

23-3

解壓縮 (Unzip) 檔案

最簡解壓縮範例

```
1  try (ZipInputStream zis =  
2      new ZipInputStream(new FileInputStream("archive.zip"))) {  
3      ZipEntry entry;  
4      while ((entry = zis.getNextEntry()) != null) {  
5          Path outputPath = Path.of("output", entry.getName());  
6          Files.createDirectories(outputPath.getParent());  
7          Files.copy(zis, outputPath);  
8          zis.closeEntry();  
9      }  
10 }
```

✓ 解壓縮三步驟：**getNextEntry()** (取得下一個 entry) → 讀取資料 → **closeEntry()** (結束此 entry)

用 ZipInputStream 解壓縮 – 完整版

```
1  public static void unzip(String src, String destDir)
2      throws IOException {
3      byte[] buf = new byte[4096];
4      try (ZipInputStream zis =
5          new ZipInputStream(new FileInputStream(src))) {
6          ZipEntry entry;
7          while ((entry = zis.getNextEntry()) != null) {
8              File out = new File(destDir, entry.getName());
9              if (entry.isDirectory()) {
10                 out.mkdirs();
11             } else {
12                 new File(out.getParent()).mkdirs();
13                 try (FileOutputStream fos = new FileOutputStream(out))
14                 {
15                     int len;
16                     while ((len = zis.read(buf)) > 0)
17                         fos.write(buf, 0, len);
18                 }
19                 zis.closeEntry();
20             }
21         }
22     }
```


Zip Slip 安全漏洞防護

```
1 // 危險：entry 名稱可能含 "../" 跳脫到目標目錄之外！
2 File out = new File(destDir, entry.getName()); // ← 未驗證
3
4 // 安全寫法：驗證輸出路徑在目標目錄內
5 File destFile = new File(destDir, entry.getName());
6 String destDirPath = new File(destDir).getCanonicalPath();
7 String destFilePath = destFile.getCanonicalPath();
8 if (!destFilePath.startsWith(destDirPath + File.separator)) {
9     throw new IOException("Zip Slip detected: " + entry.getName());
10 }
```

🔒 **Zip Slip 攻擊**：惡意 ZIP 可在 entry 名稱中使用 `../.. /etc/passwd` 等路徑跳脫到系統敏感目錄。生產環境務必加上路徑驗證。

用 ZipFile 讀取 ZIP 檔案

方法	說明
<code>ZipFile(String name)</code>	開啟指定路徑的 ZIP 檔案
<code>getEntry(String name)</code>	依名稱取得指定 entry (找不到回傳 null)
<code>getInputStream(ZipEntry e)</code>	取得指定 entry 的 InputStream
<code>entries()</code>	取得所有 entry 的 <code>Enumeration</code>
<code>stream()</code>	取得所有 entry 的 <code>Stream<ZipEntry></code> (Java 8+)
<code>size()</code>	ZIP 內的 entry 總數

ZipFile 使用範例

```
1  try (ZipFile zf = new ZipFile("archive.zip")) {
2      // 1. 列出所有 entry
3      zf.stream()
4          .map(ZipEntry::getName)
5          .forEach(System.out::println);
6
7      // 2. 讀取特定 entry 的內容
8      ZipEntry entry = zf.getEntry("data/report.csv");
9      if (entry != null) {
10         try (InputStream is = zf.getInputStream(entry)) {
11             String content = new String(is.readAllBytes());
12             System.out.println(content);
13         }
14     }
15 }
```

現代寫法

NIO + ZIP File System (Java 11+)

NIO ZIP File System 概念

```
1  FileSystems.newFileSystem(zipPath, Map.of("create", "true"))
2      ↓
3  ZipFileSystem (實作 FileSystem 介面)
4      ↓
5  可用 Files.copy / Files.walk / Files.readAllBytes
6  等所有 NIO Files 工具操作 ZIP 內容
```

💡 **ZIP File System Provider**：內建於 JDK (**jdk.zipfs** 模組)，不需額外相依。Java 13+ 支援直接傳入 **Path** 物件。

NIO：建立 ZIP 並加入檔案

```
1  Path zipPath = Path.of("output.zip");
2  Map<String, String> env = Map.of("create", "true");
3
4  try (FileSystem fs = FileSystems.newFileSystem(zipPath, env)) {
5      // 加入單一檔案
6      Files.copy(Path.of("hello.txt"), fs.getPath("hello.txt"));
7
8      // 加入到子目錄
9      Files.createDirectories(fs.getPath("data"));
10     Files.copy(Path.of("report.csv"),
11                 fs.getPath("data/report.csv"));
12 }
```

NIO：解壓縮所有檔案

```
1  Path zipPath = Path.of("archive.zip");
2  Path outDir  = Path.of("extracted");
3
4  try (FileSystem fs = FileSystems.newFileSystem(zipPath, Map.of())) {
5      Path root = fs.getPath("/");
6      Files.walk(root)
7          .filter(p → !Files.isDirectory(p))
8          .forEach(p → {
9              Path target = outDir.resolve(
10                  root.relativize(p).toString());
11              try {
12                  Files.createDirectories(target.getParent());
13                  Files.copy(p, target,
14                      StandardCopyOption.REPLACE_EXISTING);
15              } catch (IOException e) {
16                  throw new RuntimeException(e);
17              }
18          });
19 }
```


NIO vs 傳統寫法比較

比較項目	傳統 (ZipOutputStream/InputStream)	NIO ZIP File System
API 風格	IO Stream 導向	Path / Files 導向
程式碼量	較多 (手動 buffer)	較少 (Files.copy 一行)
支援版本	Java 1.1+	Java 7+ (完整 Java 11+)
串流來源	任何 InputStream	僅磁碟檔案
修改已存在 ZIP	需重新建立	可直接新增/刪除 entry
適合場景	串流處理、記憶體 ZIP	本機檔案操作

練習一：壓縮指定目錄

任務說明

撰寫一個 Java 程式，接受命令列參數：`java ZipTool <來源目錄> <輸出.zip>`

需求：

1. 遞迴壓縮來源目錄下的所有檔案與子目錄
2. ZIP 內保留目錄結構（如 `logs/2024/app.log`）
3. 使用 `try-with-resources` 確保資源正確釋放
4. 若來源目錄不存在，印出錯誤訊息並結束

練習一：解題提示

```
1  Path srcDir = Path.of(args[0]);
2  try (ZipOutputStream zos =
3      new ZipOutputStream(new FileOutputStream(args[1]))) {
4      Files.walk(srcDir)
5          .filter(p → !Files.isDirectory(p))
6          .forEach(p → {
7              String entryName = srcDir.relativize(p).toString()
8                  .replace("\\", "/");
9              // putNextEntry → Files.copy(p, zos) → closeEntry
10         });
11 }
```

關鍵步驟：

1. **Files.walk(srcDir)** 取得所有子路徑
2. **srcDir.relativize(p)** 計算相對路徑作為 entry 名稱
3. Windows 路徑分隔符 **** 要替換為 **/** (ZIP 規格要求)
4. **Files.copy(p, zos)** 取代手動 buffer 迴圈

練習二：解壓縮並統計

任務說明

撰寫一個 Java 程式解壓縮 ZIP 並輸出統計報告：

```
1  java UnzipReport <archive.zip> <輸出目錄>
```

輸出格式範例：

```
1  解壓縮完成！  
2  檔案數量：15 / 目錄數量：3  
3  總原始大小：1,234,567 bytes  
4  總壓縮大小：456,789 bytes / 壓縮比：62.98%
```

****需求：****使用 `ZipFile` 讀取、統計 `getSize()` 與 `getCompressedSize()`、加入 Zip Slip 安全防護

練習二：解題提示

```
1  long totalSize = 0, compressedSize = 0;
2  int fileCount = 0, dirCount = 0;
3
4  try (ZipFile zf = new ZipFile(args[0])) {
5      for (ZipEntry e : Collections.list(zf.entries())) {
6          if (e.isDirectory()) { dirCount++; continue; }
7          fileCount++;
8          totalSize += e.getSize();
9          compressedSize += e.getCompressedSize();
10         // 解壓縮：zf.getInputStream(e) → 寫出到目標路徑
11     }
12 }
13 double ratio = 1.0 - (double) compressedSize / totalSize;
14 System.out.printf("壓縮比：%.2f%%\n", ratio * 100);
```


Q & A

課程結束

java.util.zip：壓縮、解壓縮、安全防護一次搞定